Table des matières

Synthèse — Comptoir Aurore, une boutique en ligne de bout en bout 1
Étape 0 — La boutique, et ce que vous allez bâtir 1
Étape 1 — Générer, charger, monter le projet 2
Étape 2 — La constellation, sur papier 4
Étape 3 — Le STAGING et ses sources 4
Étape 4 — L'INTERMEDIATE : ce qui est une vente, et ce qu'elle vaut 6
Étape 5 — Les MARTS, dans trois schémas 9
Étape 6 — Les preuves 11
Étape 7 — Répondre, d'une étoile à l'autre 14
Récapitulatif 16
Annexe — L'arborescence et les commandes 18

Synthèse — Comptoir Aurore, une boutique en ligne de bout en bout

Synthèse avancée — Formation Analytics Engineering · l'arc des Cours 01 à 07, d'un bloc

Étape 0 — La boutique, et ce que vous allez bâtir

Comptoir Aurore inc. est une boutique en ligne de Montréal. Elle vend des articles de maison, d'électronique et de plein air au Canada depuis 2020, et en France depuis le 4 mars 2024. Deux marchés, deux devises, deux façons d'afficher un prix : hors taxes (HT) au Canada, toutes taxes comprises (TTC) en France. Sa base de vente est une vraie base normalisée, dix-huit tables dans PostgreSQL, et c'est elle que vous recevez, copiée au soir du 31 décembre 2025.

Là où La Brûlerie tenait dans une caisse, Aurore réunit ce qui rend un projet réel difficile. Les dates sont des instants, sans jour. Une remise porte sur toute la commande et se répartit sur les lignes. Un code promotionnel n'est unique qu'avec son marché. Des commandes payées sont annulées, des articles reviennent, et un euro ne vaut pas le même prix tous les jours. Vous allez refaire l'arc de la formation d'un bloc, sur ce décor :

la copie de la base → schéma `boutique`, et les taux de la Banque du Canada → schéma `banque_du_canada` → `STAGING`, onze vues → `INTERMEDIATE`, deux modèles → `MARTS`, dans trois schémas : les dimensions conformes dans `marts`, les faits dans `ventes` et `retours`.

© 2026 iAlexMG. Tous droits réservés.

Processus	Fait	Grain	Schéma
Un client passe une commande	`fct_commandes`	une commande vendue	`ventes`
La commande porte des produits	`fct_lignes_commande`	une ligne d'une commande vendue	`ventes`
Un client renvoie un article	`fct_retours`	un retour	`retours`

Ce document ne réexplique rien : chaque étape nomme la section de cours qui fonde son geste, et ne garde que ce qui est propre à Aurore. Il se fait après le mini-cas de La Brûlerie, une fois le Cours 07 derrière vous. Comptez deux séances, coupées après l'étape 4 : la première monte les données et la logique, la seconde l'étoile et ses preuves.

Aurore produit d'autres sources, que cette synthèse laisse à leur formation : le journal de capture des changements de la base à Data Engineering, l'analytique web à Data Engineering et à Business Intelligence, les avis des clients à AI Engineering. Les expéditions restent dans la base, et leurs délais appartiennent à Business Intelligence.

Le corrigé est dans `Solution/02_ae/`. Chaque bloc de ce document qui appartient au projet en est tiré, sans ses commentaires d'en-tête, qui disent ce que chaque fichier fait et ce qui a été mesuré.

Étape 1 — Générer, charger, monter le projet

La `RAW` reçue et sa vérification sont au Cours 01 (`Theorie_01_AE.md`, section 5). Le projet, le profil et la macro des schémas au Cours 03 (`Theorie_03_AE.md`, sections 2, 3 et 5).

Depuis `Data_Domain`, dans l'Anaconda Prompt, env `formations` actif :

```
mkdir Projet_Aurore
python _univers/aurore/generer_aurore.py --projet Projet_Aurore
```

Le générateur refuse d'écrire dans un dossier qui n'existe pas : le projet se crée d'abord. Attendu, en 45 secondes environ : 2 256 fichiers, et la ligne « Conforme à la référence des cours ». La base est dans `Projet_Aurore/sources/appli/postgresql/` : un CSV par table, et le script qui les charge, `charger_appli.sql`.

1.1 Un script de PostgreSQL, dans DuckDB

Ce script est écrit pour `psql`. Charger la `RAW` est le métier de Data Engineering, et vous n'avez pas à le réinventer : il suffit de le porter. Le DDL de PostgreSQL passe tel quel dans DuckDB, types compris ; seules trois lignes du script lui sont étrangères.

Dans `charger_appli.sql`	Dans DuckDB
`\set ON_ERROR_STOP on`, une méta-commande de `psql`	supprimée

© 2026 iAlexMG. Tous droits réservés.

Dans `charger_appli.sql`	Dans DuckDB
`ALTER TABLE boutique.produit REPLICA IDENTITY FULL;`, un réglage de la capture des changements	supprimée
`\copy boutique.categorie FROM 'donnees/categorie.csv' WITH (FORMAT csv, HEADER true, ENCODING 'UTF8')`	`COPY boutique.categorie FROM 'donnees/categorie.csv' (FORMAT csv, HEADER true);`

Le script porté, `charger_raw.sql`, ajoute un second schéma pour les taux de la Banque du Canada. Il vit dans votre dépôt, `Projet_Aurore/02_ae`, et le corrigé en fournit une copie. Il se lance depuis le dossier des CSV, parce que ses chemins sont relatifs :

```
cd Projet_Aurore\sources\appli\postgresql
duckdb ../../../aurore.duckdb -c ".read ../../../02_ae/charger_raw.sql"
```

Attendu : les 18 tables aux comptes de `_univers/aurore/CHIFFRES_ATTENDUS.md`, section 2, et 767 jours de taux, en une seconde et demie.

1.2 Le projet

Le projet `aurore_ae` vit dans `Projet_Aurore/02_ae`, un dépôt Git à lui, avec un troisième profil dans votre `profiles.yml` : la forme de celui du mini-cas, le nom `aurore_ae` et le chemin de `Projet_Aurore/aurore.duckdb`. Le `.gitignore`, le `packages.yml` et la macro `generate_schema_name` sont ceux du mini-cas. La nouveauté est dans le bloc `models:` : un dossier, un schéma.

```
models:
  aurore_ae:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      communes:
        +schema: marts
      ventes:
        +schema: ventes
      retours:
        +schema: retours
```

Rien n'oblige à découper ainsi, et c'est ce qui en fait une décision. Un domaine qui a son schéma a un propriétaire et des droits à lui, et une dimension qui n'appartient à aucun domaine se partage sans que personne ne la recopie. La macro des schémas compte ici plus que chez Boréal : sans elle, les faits atterriraient dans `staging_ventes` et `staging_retours`.

© 2026 iAlexMG. Tous droits réservés.

Étape 2 — La constellation, sur papier

La recette est au Cours 02 (`Theorie_02_AE.md`, sections 2 à 4), les dimensions conformes et la matrice de bus aux sections 9 et 10.

Dimension	fct_commandes	fct_lignes_commande	fct_retours
dim_date	oui	oui	oui, deux fois
dim_client	oui	oui	oui
dim_produit	non	oui	oui
dim_promotion	oui	non	non

Trois décisions portent tout le reste.

2.1 Une vente, deux grains

Les frais de livraison, les taxes, le total payé et le taux de change appartiennent à la commande. Le produit, la quantité et le rabais de fin de série appartiennent à la ligne. Répartir les premiers sur les lignes les ferait tomber à côté de leur total, comme les taxes de La Brûlerie. Deux faits, donc : l'un au grain de la commande, l'autre au grain de la ligne, et un test qui prouve qu'ils disent le même chiffre.

La remise d'un code promotionnel est le cas limite. Elle porte sur la commande, et l'analyse par produit la veut sur les lignes : elle se répartira, à l'étape 4, par une méthode qui retombe juste.

2.2 Ce qui est une vente

La base compte 257 commandes annulées : 140 dont le paiement a été refusé, 117 payées puis annulées par le client et remboursées en entier. Aucune n'est une vente, et elles sortent des deux faits. La règle s'écrit une seule fois, dans la couche `INTERMEDIATE`.

2.3 Le jour d'une commande

La base ne connaît que des instants, en UTC, le temps universel coordonné, sans heure d'été. Le jour d'une commande est donc un choix : celui de Montréal, parce que c'est le calendrier de la boutique. Les prix changent à minuit à Montréal, et un test le prouvera à l'étape 6. Avec ce choix, la période tombe juste, du 1er janvier 2023 au 31 décembre 2025 ; en UTC, les commandes du soir du 31 décembre 2025 déborderaient sur 2026.

`dim_date` ne change pas pour autant : c'est celle du Cours 04, et ce sont les faits qui choisissent quel jour ils y rangent.

Étape 3 — Le `STAGING` et ses sources

© 2026 iAlexMG. Tous droits réservés.

Les sources sont au Cours 03 (`Theorie_03_AE.md`, section 4), les quatre gestes à la section 6, la sévérité et le seuil d'un test au Cours 05 (`Theorie_05_AE.md`, section 4), et ce qu'une dimension de personnes a le droit de porter au Cours 06 (`Theorie_06_AE.md`, section 6).

Le fichier `_sources.yml` déclare deux sources, `boutique` et `banque_du_canada`, et seulement les tables qu'un modèle ou un test lit : douze des dix-huit, plus les taux. Un seul test y vit, et il dit un défaut de la source plutôt que du projet :

```yaml
      - name: commande
        columns:
          - name: adr_livraison_id
            data_tests:
              - relationships:
                  arguments:
                    to: source('boutique', 'adresse')
                    field: adr_id
                  config:
                    severity: warn
                    warn_if: ">20"
```

Un client peut supprimer une adresse, et la ligne disparaît même si une commande la cite. Mesuré : 20 commandes, dont les 3 de la cliente de France effacée à sa demande le 4 novembre 2025. Le seuil les laisse passer en silence et signale la 21e. Un test qui échouerait tous les jours ne dirait plus rien à personne.

Onze vues, une par table qu'un modèle lit. Chez Aurore, les quatre gestes du `STAGING` trouvent chacun leur travail, et trois points demandent un mot.

3.1 Le fuseau que DuckDB choisit pour vous

Une colonne `timestamptz` n'a pas de jour : pour en tirer un, il faut un fuseau, et DuckDB prend celui du poste si vous ne le donnez pas. L'heure de mur, l'instant et le fuseau de session sont au Cours 02 de la formation SQL avancé, section 6. Mesurez l'écart, dans la ligne de commande DuckDB lancée depuis `Projet_Aurore/` :

```sql
select
    count(*) as commandes,
    count(*) filter (
        where (cree_le at time zone 'UTC')::date
           <> (cree_le at time zone 'America/Montreal')::date
    ) as jour_different
from boutique.commande;
```

Sur 11 599 commandes, 3 561 changent de jour entre l'UTC et Montréal, et 138 changent de mois. Le fuseau s'écrit donc en toutes lettres, dans le modèle qui tire le jour :

```sql
select
    cmd_id,
    cli_id,
```


© 2026 iAlexMG. Tous droits réservés.

```
    adr_livraison_id,
    marche,
    devise,
    statut,
    code_promo,
    coalesce(mode_livraison, 'DOMICILE') as mode_livraison,
    sous_total,
    remise_commande,
    frais_livraison,
    taxes,
    total,
    cree_le,
    (cree_le at time zone 'America/Montreal')::date as jour_commande

from {{ source('boutique', 'commande') }}
```

Le `coalesce` est un décodage, pas un bouche-trou. Le mode de livraison est une colonne ajoutée le 20 janvier 2025, vide avant, et avant cette date la livraison à domicile était la seule qui existait. Le vide voulait dire « à domicile », pas « inconnu ».

3.2 Un code qui n'est unique qu'avec son marché

La table des promotions porte 19 lignes pour 16 codes : `BIENVENUE10` et les deux codes du vendredi fou existent des deux côtés de l'Atlantique. La jointure qui semble évidente dédouble les commandes :

```
select count(*)
from boutique.commande as c
inner join boutique.promotion as p on p.code = c.code_promo;
```

Elle rend 4 891 lignes pour 2 642 commandes à code. La clé de la promotion est le couple, code et marché, et elle le restera jusqu'au fait.

3.3 Ce que les clients ne montent pas

`stg_clients` garde le marché, la langue, le segment, la date de création du compte, et rien qui permette d'écrire à quelqu'un. Le courriel sert une seule fois, pour dire si le compte a été effacé : un client effacé garde son identifiant, et ses renseignements sont vidés. La cliente effacée en novembre 2025 reste donc une ligne, avec son drapeau, et ses trois commandes gardent un client.

Étape 4 — L'`INTERMEDIATE` : ce qui est une vente, et ce qu'elle vaut

La couche `INTERMEDIATE` est au Cours 04 (`Theorie_04_AE.md`, section 6). La jointure ASOF est au Cours 09 de la formation SQL avancé, section 6, et les fonctions de fenêtre à son Cours 04 ; ici, on les emploie sans les réexpliquer.

© 2026 iAlexMG. Tous droits réservés.

Deux modèles `ephemeral`, et chacun tient une règle qui ne doit s'écrire qu'une fois.

4.1 Les commandes vendues, au taux du jour

```
select
    c.cmd_id,
    c.cli_id,
    c.marche,
    c.devise,
    c.code_promo,
    c.mode_livraison,
    c.sous_total,
    c.remise_commande,
    c.frais_livraison,
    c.taxes,
    c.total,
    c.jour_commande,
    case when c.devise = 'EUR' then t.eur_cad end as taux_eur_cad

from {{ ref('stg_commandes') }} as c
asof left join {{ ref('stg_taux_change') }} as t
    on c.jour_commande >= t.jour

where c.statut <> 'ANNULEE'
```

Le filtre sur le statut est la règle de l'étape 2.2, et les deux faits de vente lisent ce modèle sans la répéter. Une règle écrite à deux endroits finit par dire deux choses.

La jointure ASOF règle le problème du taux. La Banque du Canada ne publie rien la fin de semaine ni les jours fériés, et 613 des 2 028 ventes en euros tombent un de ces jours. Pour chaque commande, la jointure prend le taux du jour le plus récent qui ne la dépasse pas : le vendredi pour une commande du samedi. Elle est à gauche, pour qu'une commande sans taux reste visible, et un test sur le fait vérifiera qu'aucune ne l'est.

4.2 La remise, répartie sans perdre un cent

La règle naturelle donne à chaque ligne sa part de la remise, au prorata de son montant, arrondie au cent. Elle ne retombe pas sur le total : mesuré, 87 des 2 580 commandes vendues avec un code gagnent ou perdent un cent ou deux, 0,48 $ et 0,28 € en tout. La méthode cumulative retombe juste par construction. Elle s'écrit en expressions de table communes, au Cours 05 de la formation SQL avancé, sections 2 et 3, et en cumul par fenêtre, à son Cours 04, section 5 :

```
with lignes as (

    select
        l.cmd_id,
```


© 2026 iAlexMG. Tous droits réservés.

```
        l.lig_no,
        l.prod_id,
        l.quantite,
        l.remise_ligne,
        l.montant,
        c.remise_commande,
        c.sous_total,
        sum(l.montant) over (
            partition by l.cmd_id
            order by l.lig_no
        ) as cumul

    from {{ ref('stg_commande_lignes') }} as l
    inner join {{ ref('int_commandes_vendues') }} as c
        on c.cmd_id = l.cmd_id

)

select
    cmd_id,
    lig_no,
    prod_id,
    quantite,
    remise_ligne,
    montant,
    round(remise_commande * cumul / sous_total, 2)::decimal(12, 2)
        - round(remise_commande * (cumul - montant) / sous_total, 2)::decimal(12, 2)
        as part_remise_commande

from lignes
```

On arrondit la remise cumulée jusqu'à chaque ligne, et la part d'une ligne est la différence entre deux cumuls arrondis. La somme se télescope : la dernière ligne reçoit exactement ce qui reste, et aucune ne s'écarte de plus d'un cent de la règle naturelle.

Piège — la division de deux décimaux rend un flottant dans DuckDB. Sans le `::decimal(12, 2)`, les parts sont des nombres à virgule flottante, et leur somme ne se compare plus exactement à un décimal : mesuré, plus de 2 000 commandes paraissent en écart, pour moins d'un milliardième de cent. Le même défaut que les seeds de La Brûlerie, arrivé par un autre chemin.

Le cas se refait de tête, et c'est ce qui en fait un bon test unitaire : dix dollars de remise sur trois lignes de dix dollars. La règle naturelle donne trois fois 3,33 $ et perd un cent ; la méthode cumulative donne 3,33 $, 3,34 $ et 3,33 $. Le test est à l'étape 6.

Ici se termine la première séance.

© 2026 iAlexMG. Tous droits réservés.

Étape 5 — Les `MARTS`, dans trois schémas

La clé de substitution est au Cours 04 (`Theorie_04_AE.md`, section 3), `dim_date` à la section 4, le fait à la section 7 ; l'étoile et le flocon au Cours 02 (`Theorie_02_AE.md`, section 5).

5.1 Les dimensions conformes

Dimension	Lignes	Ce qu'elle porte
`dim_date`	1 096	celle de Boréal, du 2023-01-01 au 2025-12-31
`dim_client`	6 680	le marché, la langue, le segment, la date de création, le drapeau d'effacement
`dim_produit`	86	le SKU (le code d'une variante vendue : une couleur ou une taille a le sien), le modèle, la catégorie, la famille, le fournisseur
`dim_promotion`	19	un code par marché, sa valeur et sa période ; la clé est hachée sur le couple

`dim_produit` aplatit la hiérarchie que la base range en une table qui se pointe elle-même :

```
select
    {{ dbt_utils.generate_surrogate_key(['p.prod_id']) }} as cle_produit,
    p.prod_id,
    p.sku,
    p.modele,
    p.nom_fr as nom,
    c.nom_fr as categorie,
    f.nom_fr as famille,
    r.nom as fournisseur,
    p.vendu_en_france,
    p.est_actif,
    p.lance_le,
    p.retire_le

from {{ ref('stg_produits') }} as p
inner join {{ ref('stg_categories') }} as c
    on c.cat_id = p.cat_id
```


© 2026 iAlexMG. Tous droits réservés.

```
inner join {{ ref('stg_categories') }} as f
    on f.cat_id = c.parent_id
inner join {{ ref('stg_fournisseurs') }} as r
    on r.four_id = p.four_id
```

Deux jointures sur la même table, la catégorie puis sa famille : c'est l'étoile contre le flocon, et le choix se paie une fois ici plutôt qu'à chaque requête. Le modèle, `AUR-2101` pour les deux couleurs d'un même écouteur, est tiré du SKU dans `stg_produits`.

5.2 Les deux faits de vente

```
select
    c.jour_commande as cle_date,
    {{ dbt_utils.generate_surrogate_key(['c.cli_id']) }} as cle_client,
    case
        when c.code_promo is not null
            then {{ dbt_utils.generate_surrogate_key(['c.code_promo', 'c.marche']) }}
    end as cle_promotion,
    c.cmd_id,
    c.marche,
    c.devise,
    c.mode_livraison,
    c.sous_total,
    c.remise_commande,
    c.frais_livraison,
    c.taxes,
    c.total,
    c.total - c.taxes as montant_ht,
    c.taux_eur_cad,
    case
        when c.devise = 'EUR'
            then round((c.total - c.taxes) * c.taux_eur_cad, 2)::decimal(12, 2)
        else c.total - c.taxes
    end as montant_ht_cad

from {{ ref('int_commandes_vendues') }} as c
```

Les montants restent dans la devise du marché, tels que le client les a vus. Une seule colonne s'additionne d'un marché à l'autre : `montant_ht_cad`, le total moins les taxes, frais de livraison compris, converti au taux du jour pour la France. La clé de la promotion est nulle sans code, pour la raison que le mini-cas a mesurée sur la clé du membre. Attendu : 11 342 commandes.

© 2026 iAlexMG. Tous droits réservés.

`fct_lignes_commande` porte, au grain de la ligne, la quantité, le montant, le rabais de fin de série, la part de la remise et le montant net, dans la devise du marché. Il ne convertit rien : converties ligne par ligne puis arrondies, les lignes ne retomberaient pas sur leur commande. Qui veut des dollars pour les deux marchés passe par `fct_commandes`. Attendu : 19 398 lignes.

5.3 Le fait des retours, et ses deux dates

```
select
    r.jour_demande as cle_date,
    c.jour_commande as cle_date_vente,
    {{ dbt_utils.generate_surrogate_key(['l.prod_id']) }} as cle_produit,
    {{ dbt_utils.generate_surrogate_key(['c.cli_id']) }} as cle_client,
    r.ret_id,
    r.cmd_id,
    r.lig_no,
    c.marche,
    c.devise,
    r.motif,
    r.statut,
    r.quantite,
    b.montant as montant_rembourse

from {{ ref('stg_retours') }} as r
inner join {{ ref('stg_commande_lignes') }} as l
    on l.cmd_id = r.cmd_id
   and l.lig_no = r.lig_no
inner join {{ ref('stg_commandes') }} as c
    on c.cmd_id = r.cmd_id
left join {{ ref('stg_remboursements') }} as b
    on b.ret_id = r.ret_id
```

Un retour a son jour, celui de la demande, et la vente qu'il défait a le sien. Les deux pointent `dim_date`, sous deux noms : c'est la seconde qui permettra de demander quelle part des ventes d'une période est revenue. Le montant rendu reste vide pour les 55 retours refusés ou en cours. Attendu : 980 retours, dont 925 remboursés.

Étape 6 — Les preuves

Les tests génériques sont au Cours 05 (`Theorie_05_AE.md`, section 2), la réconciliation en jointure externe complète à la section 3, le test unitaire à la section 5.

© 2026 iAlexMG. Tous droits réservés.

Les trois fichiers YAML des `MARTS` posent 32 tests génériques, la source un 33e, et trois tests singuliers portent le compte à 36. Les règles sont celles de Boréal : l'unicité et la présence de chaque clé, le grain de chaque fait, une `relationships` par clé de dimension, un `accepted_values` jamais seul. Deux s'ajoutent ici : le grain de `dim_promotion` se teste sur le couple code et marché, et un `not_null` sur `montant_ht_cad` témoigne de la jointure ASOF.

6.1 Les ventes contre l'argent encaissé

C'est le cœur de la synthèse. Le fait part des commandes ; les paiements viennent du prestataire. Pour chaque commande vendue, le total doit égaler ce qui a été encaissé, net de ce qui a été rendu à l'annulation :

```
with fait as (

    select cmd_id, total
    from {{ ref('fct_commandes') }}

),

paye as (

    select cmd_id, sum(montant) as montant
    from {{ ref('stg_paiements') }}
    where statut = 'AUTORISE'
    group by 1

),

rendu as (

    select cmd_id, sum(montant) as montant
    from {{ ref('stg_remboursements') }}
    where motif = 'ANNULATION'
    group by 1

),

encaisse as (

    select paye.cmd_id, paye.montant - coalesce(rendu.montant, 0) as montant
    from paye
    left join rendu on rendu.cmd_id = paye.cmd_id

)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
select
    coalesce(fait.cmd_id, encaisse.cmd_id) as cmd_id,
    fait.total as total_du_fait,
    encaisse.montant as encaisse_net

from fait
full outer join encaisse on encaisse.cmd_id = fait.cmd_id

where (fait.cmd_id is null and encaisse.montant <> 0)
   or encaisse.cmd_id is null
   or fait.total <> encaisse.montant
```

Il mord, et il faut le voir mordre. Retirez le filtre sur le statut à l'étape 4.1 : les annulées entrent dans le fait, et le test échoue sur les 257. Remplacez ensuite `full outer join` par `inner join` : il n'en voit plus que 117, celles qui ont un paiement. Les 140 jamais payées n'ont rien à joindre de l'autre côté, et passent.

À retenir — c'est l'erreur que l'ancienne version de cette synthèse portait. Son fait comptait les commandes annulées comme des ventes, et son test de réconciliation, écrit en jointure interne, passait au vert sans les avoir jamais regardées. Une réconciliation qui ne compare que ce qui existe des deux côtés ne voit jamais ce qui manque. Le `full outer join` est au Cours 01 de la formation SQL avancé, section 7, et le rapprochement dans les deux sens à son Cours 07, section 6.

6.2 Les deux faits disent le même chiffre

`assert_lignes_egalent_commandes.sql` compare, commande par commande, la somme des montants nets des lignes au sous-total moins la remise, en jointure externe complète et au cent. Avec la règle naturelle à la place de la méthode cumulative, il échoue sur 87 commandes.

6.3 Le prix du jour, et le jour qui va avec

`assert_prix_du_jour.sql` vérifie que chaque ligne porte le prix du catalogue en vigueur le jour de la commande, pour son marché. La base garde l'historique des prix, un prix par produit, par marché et par période : une règle datée se vérifie contre une table datée. Au jour de Montréal, aucune ligne en écart. Au jour UTC, 5 : des commandes passées le soir, à Montréal, la veille d'une hausse. C'est ce test qui justifie le fuseau de l'étape 2.3.

6.4 Le test unitaire

```yaml
unit_tests:
  - name: remise_repartie_sans_perdre_un_cent
    description: >
      Dix dollars de remise sur trois lignes de dix dollars : la règle proportionnelle donne trois
      fois 3,33 $ et perd un cent ; la méthode cumulative donne 3,33 $, 3,34 $ et 3,33 $.
    model: int_lignes_nettes
    given:
      - input: ref('stg_commande_lignes')
        rows:
```


© 2026 iAlexMG. Tous droits réservés.

```
        - {cmd_id: 1, lig_no: 1, prod_id: 7, quantite: 1, remise_ligne: 0, montant: 10.00}
        - {cmd_id: 1, lig_no: 2, prod_id: 8, quantite: 1, remise_ligne: 0, montant: 10.00}
        - {cmd_id: 1, lig_no: 3, prod_id: 9, quantite: 1, remise_ligne: 0, montant: 10.00}
    - input: ref('int_commandes_vendues')
      format: sql
      rows: |
        select
            1 as cmd_id,
            10.00::decimal(12, 2) as remise_commande,
            30.00::decimal(12, 2) as sous_total
  expect:
    rows:
      - {cmd_id: 1, lig_no: 1, part_remise_commande: 3.33}
      - {cmd_id: 1, lig_no: 2, part_remise_commande: 3.34}
      - {cmd_id: 1, lig_no: 3, part_remise_commande: 3.33}
```

Piège — une entrée `ephemeral` s'écrit en SQL. dbt lit dans la base les colonnes de chaque entrée d'un test unitaire, et un modèle `ephemeral` n'y existe pas. Écrite comme la première, en lignes, l'entrée `int_commandes_vendues` fait échouer le test sur « the relation doesn't exist ». Le `format: sql` donne les colonnes et leurs types d'un coup.

Avec la règle naturelle, c'est ce test qui tombe le premier, sur trois lignes qu'on recompte de tête, avant qu'aucune donnée ne soit construite.

Tout se lance d'un geste, depuis `Projet_Aurore/02_ae` :

```
dbt build
```

Attendu : `PASS=55 WARN=0 ERROR=0`, soit 11 vues, 7 tables, 36 tests de données et 1 test unitaire, en un peu plus de deux secondes d'exécution.

Étape 7 — Répondre, d'une étoile à l'autre

Le forage transversal est au Cours 02 (`Theorie_02_AE.md`, section 12), et son usage sur un incident au Cours 06 (`Theorie_06_AE.md`, section 12). La documentation, `dbt docs generate`, est au Cours 05, section 9, et le graphe y va des deux schémas sources aux trois schémas de l'étoile. L'agrégat filtré, `filter (where …)`, qui range chaque marché dans sa colonne, est au Cours 06 de la formation SQL avancé, section 6.

7.1 Les ventes, par année et par marché

```
select
    d.annee,
    sum(f.sous_total - f.remise_commande) filter (where f.marche = 'CA') as ventes_ca_ht,
    sum(f.total) filter (where f.marche = 'FR') as ventes_fr_ttc,
```


© 2026 iAlexMG. Tous droits réservés.

```
    sum(f.montant_ht_cad) filter (where f.marche = 'FR') as ventes_fr_ht_cad
from ventes.fct_commandes as f
inner join marts.dim_date as d on d.date_jour = f.cle_date
group by d.annee
order by d.annee;
```

Année	Ventes au Canada, hors taxes	Ventes en France, TTC	Ventes en France, HT, en dollars
2023	537 658,57 $	—	—
2024	661 420,30 $	122 717,22 €	152 087,85 $
2025	788 525,58 $	217 725,14 €	286 767,65 $

`CHIFFRES_ATTENDUS.md` compte par année UTC, et trois comparaisons disent trois choses. Les ventes en France, taxes comprises, tombent au cent sur les siennes. Les ventes au Canada diffèrent d'une année à l'autre, parce que les commandes du soir du 31 décembre changent d'année selon le fuseau ; sur toute la période, les deux comptes donnent 1 987 604,45 $. Et les ventes françaises converties s'écartent de 5,75 $ en 2024 et de 0,83 $ en 2025 : le générateur prend le taux du jour UTC, le projet celui du jour de Montréal, et une commande du soir ne reçoit pas le même taux. Un montant converti porte sa convention de date, et ne se compare qu'à un montant converti de la même façon.

7.2 Le lot d'écouteurs défectueux

Des écouteurs sans fil de première version, le modèle `AUR-2101`, sont vendus du 1er novembre 2024 au 10 janvier 2025 avec un défaut, puis retirés. La question se pose d'une étoile à l'autre : quelle part des lignes vendues dans cette fenêtre est revenue pour défaut. Chaque fait s'agrège de son côté, sur les dimensions qu'ils partagent, puis les deux se joignent :

```
with vendues as (

    select
        d.date_jour between date '2024-11-01' and date '2025-01-10' as dans_la_fenetre,
        count(*) as lignes
    from ventes.fct_lignes_commande as f
    inner join marts.dim_produit as p on p.cle_produit = f.cle_produit
    inner join marts.dim_date as d on d.date_jour = f.cle_date
    where p.modele = 'AUR-2101'
    group by 1

),

revenues as (

    select
        d.date_jour between date '2024-11-01' and date '2025-01-10' as dans_la_fenetre,
```


© 2026 iAlexMG. Tous droits réservés.

```
        count(*) as retours_pour_defaut
    from retours.fct_retours as r
    inner join marts.dim_produit as p on p.cle_produit = r.cle_produit
    inner join marts.dim_date as d on d.date_jour = r.cle_date_vente
    where p.modele = 'AUR-2101'
      and r.motif = 'DEFECTUEUX'
    group by 1

)

select
    v.dans_la_fenetre,
    v.lignes,
    r.retours_pour_defaut,
    round(100.0 * r.retours_pour_defaut / v.lignes, 1) as pour_cent
from vendues as v
left join revenues as r on r.dans_la_fenetre = v.dans_la_fenetre
order by v.dans_la_fenetre desc;
```

Vendues	Lignes	Retours pour défaut	Part
dans la fenêtre	59	26	44,1 %
hors de la fenêtre	300	8	2,7 %

Le retour se range sur le jour de la vente, par `cle_date_vente`. Rangé sur le jour de la demande, un écouteur vendu en décembre et renvoyé en février tomberait hors de la fenêtre : la question porte sur ce qui a été vendu, et c'est la date de la vente qui y répond. Le générateur compte 8 retours sur 306 lignes hors de la fenêtre, 2,6 % : ses 306 comprennent les 6 lignes de commandes annulées, que le fait n'a pas.

Récapitulatif

Démarche	Où elle est enseignée	Où ici
la `RAW` reçue, montée et comptée	Cours 01, section 5	Étape 1.1
le projet, le profil, la macro des schémas	Cours 03, sections 2, 3 et 5	Étape 1.2
un dossier, un schéma	cette synthèse	Étape 1.2

© 2026 iAlexMG. Tous droits réservés.

Démarche	Où elle est enseignée	Où ici
la recette, les dimensions conformes, la matrice de bus	Cours 02, sections 2 à 4, 9 et 10	Étape 2
deux grains pour une vente	Cours 02, section 3 ; cette synthèse	Étape 2.1
les sources, et un test posé sur l'une d'elles	Cours 03, section 4 ; Cours 05, section 4	Étape 3
le jour d'un instant, et le fuseau	cette synthèse	Étapes 2.3, 3.1 et 6.3
la minimisation des données personnelles	Cours 06, section 6	Étape 3.3
l'`INTERMEDIATE`, une règle écrite une fois	Cours 04, section 6	Étape 4.1
le taux d'un jour sans publication	SQL avancé, Cours 09, section 6	Étape 4.1
la remise répartie sans perdre un cent	cette synthèse	Étape 4.2
les clés fabriquées, la hiérarchie aplatie	Cours 04, section 3 ; Cours 02, section 5	Étape 5.1
une date qui joue deux rôles	cette synthèse	Étape 5.3
les tests génériques, la réconciliation, le test unitaire	Cours 05, sections 2, 3 et 5	Étape 6
le forage d'une étoile à l'autre	Cours 02, section 12 ; Cours 06, section 12	Étape 7.2

Ce que la synthèse ne fait pas, et pourquoi :

Ce qui manque	La raison
le snapshot	la base garde elle-même l'historique qui compte, celui des prix, et la copie n'a qu'un état
l'incrémental	la copie de la base est un état complet : elle se recharge, elle ne s'allonge pas
le journal de capture, le web, les avis	ils appartiennent à Data Engineering, à Business Intelligence et à AI Engineering
les expéditions	leurs délais sont la matière de Business Intelligence

Aurore est Boréal sans filet : une base qu'on n'a pas dessinée, deux marchés qui ne comptent pas pareil, et des chiffres qui ne tombent juste que si chaque convention est écrite, le jour, le taux, l'arrondi. Chacune se prouve par un test qui a été vu échouer.

© 2026 iAlexMG. Tous droits réservés.

Annexe — L'arborescence et les commandes

```
Projet_Aurore/
├─ sources/                          la sortie du générateur
├─ aurore.duckdb                     la base, hors du dépôt
└─ 02_ae/                            le dépôt de la synthèse
   ├─ charger_raw.sql                le chargement, porté de psql à DuckDB
   ├─ dbt_project.yml                un dossier, un schéma
   ├─ packages.yml
   ├─ .gitignore
   ├─ macros/generate_schema_name.sql
   ├─ models/
   │  ├─ staging/                    _sources.yml et onze vues stg_*
   │  ├─ intermediate/               int_commandes_vendues, int_lignes_nettes
   │  └─ marts/
   │     ├─ communes/                dim_date, dim_client, dim_produit, dim_promotion
   │     ├─ ventes/                  fct_commandes, fct_lignes_commande, le test unitaire
   │     └─ retours/                 fct_retours
   └─ tests/                         les trois tests singuliers
```

Pour tout rejouer de zéro, depuis `Data_Domain`, env `formations` actif :

```
mkdir Projet_Aurore
python _univers/aurore/generer_aurore.py --projet Projet_Aurore
cd Projet_Aurore\sources\appli\postgresql
duckdb ../../../aurore.duckdb -c ".read ../../../02_ae/charger_raw.sql"
cd ..\..\..\02_ae
dbt deps
dbt build
```

Le corrigé complet, commentaires compris, est dans `Solution/02_ae/`. Son `README.md` dit comment le rejouer sans rien retaper.

© 2026 iAlexMG. Tous droits réservés.